



Processing a PHP form 60m

To process a POST request in PHP, you need to use the **\$_POST** variable, one of the built-in PHP superglobals, which are variables that are automatically available to you in a PHP web page. The syntax for the **\$_POST** is:



Copy

```
1 $_POST["variable_name"];
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)

This represents an associative array of the variables from an HTML form, passed to the current script via the HTTP POST method. The names of these variables come from the 'name' attributes of the data fields on the form.

NOTE: The HTML 'name' attribute is an essential part of each HTML element here. The 'id' attribute is only useful for CSS and client-side JavaScript and does not relate to the form data.

PHP can be added to the body of the document using PHP tags.
[Skip to main content](#)

In this example, you are going to echo some data back to the same page, so the position of the PHP tags will affect where that data appears.

The first thing to do is to check that a POST request has been made (that is, the 'Submit' button on the form has been pressed). To do this, you use the code:



Copy

```
1 if ($_SERVER["REQUEST_METHOD"] == "POST")
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)

The first thing to do is to check that a POST request has been made (that is, the 'Submit' button on the form has been pressed). To do this, you use the code:



Copy

```
1 $firstname = $_POST["firstname"];
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)

Once the data has been extracted from the request, you can perform operations on it. In Topic 10, we will look at sending this data into a database, but for this example, we will just echo the values back onto the same web page using suitable HTML tags:

[Skip to main content](#)



Copy

```
1 echo "<h2>First name: $firstname</h2>";
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)

Below is the complete code for the 'registercustomer.php' page. Save the code, then execute the code on your web server to view and interact with the customer registration form. Test that the code works by filling in the form and then selecting the 'Register' button.



Copy

[Skip to main content](#)

```
1  <!DOCTYPE HTML>
2  <html><head><title>Register new customer</title> </head>
3  <body>
4    <h1>New Customer Registration</h1>
5    <form method="POST" action="<?php echo $_SERVER['PHP_SELF'];?>">
6      <p>
7        <label for="firstname">Name: </label>
8        <input type="text" id="firstname" name="firstname" minlength="5" maxlength="50" re
9      </p>
10     <p>
11       <label for="lastname">Last Name: </label>
12       <input type="text" id="lastname" name="lastname" minlength="5" maxlength="50" requ
13     </p>
14     <p>
15       <label for="email">Email: </label>
16       <input type="email" id="email" name="email" minlength="5" maxlength="50" required>
17     </p>
18     <p>
19       <label for="password">Password: </label>
20       <input type="password" id="password" name="password" minlength="8" maxlength="32"
21     </p>
22     <input type="submit" name="submit" value="Register">
23   </form>
24   <?php
25   //check if there is posted data
26   if ($_SERVER["REQUEST_METHOD"] == "POST")
27   {
```



[Skip to main content](#)

Note: Notice the 'action' attribute for the form? You will see that in this example, instead of hard-coding in the name of the PHP page (for example, <form method="POST" action="registercustomer.php">), we've instead used the `PHP_SELF` element of the `$_SERVER` array, shown below:



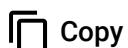
Copy

```
1 <form method="POST" action="<?php echo
2 htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)

It is useful to do this because it means the code won't break if the file is renamed. However, it also opens up the page to potential hacking. To prevent this, the **htmlspecialchars** function should be used. This converts special HTML characters like < and > to <; and >, preventing attackers from injecting malicious HTML or JavaScript code. The 'action' attribute should therefore be modified to include this function:



Copy

```
1 <form method="POST" action="<?php echo
2 htmlspecialchars($_SERVER["PHP_SELF"]);?>">
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)[Skip to main content](#)



In the previous example, we saw how an HTML page can contain a simple form that gathers customer details using the POST request. In this example, data that was formatted as text or type information was passed to the web server. Well, it is also possible to send or pass other formats of data to the web server, such as word-processed documents, image files, PDFs, videos and ZIP files. The case study below will illustrate how to upload a file to the server using PHP script.

Joe is from a leading media company. He approaches you asking for your guidance on how to handle different types of image files for his new project. Currently, his client emails him image files that are in different formats. The problems Joe is facing are:

- Once he receives the image files from his client, he has to convert them to a valid format. The valid formats are PNG, JPEG, JPG, BMP or GIF.
- He is downloading and sharing the images with his team, which is time-consuming.
- He is misplacing the image files in different folders.
- Create an HTML form where the client can upload the image files. This HTML form will allow the client to browse their files by using the choose file button. Then, once they've selected their image file, they can submit their chosen image by using the submit button.
- Use PHP to check the image file extensions to ensure the image files submitted are in a valid format. To check that the submitted image files have a valid extension, you need to use the PHP code:

Joe needs an application where his clients can only upload valid image files, so images that are in invalid formats cannot be sent to Joe. The image files can then be saved in a particular location of the web server where they can be accessed and utilised by the team for project purposes.

[Skip to main content](#)

To solve Joe's problems, he can use an HTML form that allows the client to upload and submit their image files. To do this, Joe would need to follow the steps below.

Step 1

Create an HTML form where the client can upload the image files. This HTML form will allow the client to browse their files by using the choose file button. Then, once they've selected their image file, they can submit their chosen image by using the submit button.

Step 2

Use PHP to check the image file extensions to ensure the image files submitted are in a valid format. To check that the submitted image files have a valid extension, you need to use the PHP code:



[Skip to main content](#)



Copy

```
1  if ( isset( $_FILES['imgUpload'] ) ) {  
2  if ( ( $_FILES['imgUpload']['type'] == "image/png" ) ||  
3  ( $_FILES['imgUpload']['type'] == "image/PNG" ) ||  
4  ( $_FILES['imgUpload']['type'] == "image/jpg" ) ||  
5  ( $_FILES['imgUpload']['type'] == "image/JPG" ) ||  
6  ( $_FILES['imgUpload']['type'] == "image/JPEG" ) ||  
7  ( $_FILES['imgUpload']['type'] == "image/jpeg" ) ||  
8  ( $_FILES['imgUpload']['type'] == "image/gif" ) ||  
9  ( $_FILES['imgUpload']['type'] == "image/GIF" ) ) {  
10 //...
```

gistfile1.txt hosted with ❤ by GitHub

[view raw](#)

Step 3:

The submitted image file will be sent to the temporary directory on the server while the PHP code is checking for a valid extension. To do this, use the PHP code:



[Skip to main content](#)



Copy

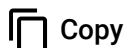
```
1 $source_file = $_FILES['imgUpload']['tmp_name'];  
2 $dest_file = "upload/".$_FILES['imgUpload']['name'];
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)

Step 4:

If the submitted image file has a valid extension, then the image file will be copied into its intended directory (in this case, the folder is called 'upload'). To do this, use the PHP code:



Copy

```
1 $dest_file = "upload/".$_FILES['imgUpload']['name'];
```

gistfile1.txt hosted with ❤️ by GitHub

[view raw](#)☒ Uploading images

Non-Assessed

[Skip to main content](#)

In this activity, you will create the PHP code to help Joe create an HTML form that will only accept image files with a valid extension and then save them in a folder called 'upload'.

Before you begin, you will need to:

- save your PHP code in a file called touplimg.php and save it in the www/projects folder of your server
- create a folder called 'upload', which will store the uploaded images.

The tasks you must perform in this activity are:

- If an image is uploaded with a valid image extension (PNG, JPEG, JPG, BMP or GIF) then the image must be copied to a folder called upload.
- If an image is uploaded with an invalid image extension (anything other than PNG, JPEG, JPG, BMP or GIF), then an error message must display letting the user know their uploaded image file has an invalid image file extension.
- If an image is uploaded that has already been uploaded previously, then an error message must display letting the user know their uploaded image has already been uploaded and is available.

☐ I have completed the task

Submit



Thoughts on this page?



[Skip to main content](#)

